

NTSEC RADAR · DOCUMENTAÇÃO TÉCNICA

Manual de Construção — Radar de Boletins

Como reconstruir o sistema do zero, com uma IA de codificação, para quem nunca programou

o. Antes de começar

Este documento ensina a construir o NTSEC Radar (módulo Boletins) inteiro, do zero, usando uma IA de codificação como o **Claude Code** como parceira de construção. Ele não presume conhecimento de programação. Presume só que você vai copiar os textos em destaque (os "prompts") e colar na conversa com a IA, e vai olhar o resultado antes de seguir para o próximo passo.

A estratégia: fases pequenas, cada uma com um jeito de verificar

Havia duas formas possíveis de organizar este manual: **(a)** um prompt único e gigante pedindo "construa o sistema inteiro", ou **(b)** uma sequência de fases pequenas, cada uma com uma forma concreta de conferir que funcionou antes de avançar. Este manual usa a **(b)**. A razão não é teórica; é a experiência real de construir e manter este sistema:

- Pedidos grandes demais fazem a IA acertar 90% e errar um detalhe pequeno que só aparece muito depois. Aí fica difícil saber qual das cem linhas mudadas causou o problema.
- Uma fase pequena com verificação imediata isola o erro no instante em que ele aparece. Se a Fase 3 quebrar, você sabe que é da Fase 3, porque as fases 1 e 2 já foram confirmadas funcionando.
- Quando algo dá errado, **a ação certa é colar a mensagem de erro exata de volta na IA**, não tentar descrever o problema com suas palavras. A IA lê o erro melhor do que lê a sua descrição dele.

VERIFICAÇÃO

Depois de cada fase deste manual, existe uma linha assim, dizendo exatamente o que olhar antes de ir para a próxima. Não pule — é isso que torna o processo seguro para quem não programa.

Por que esta stack (e não outra mais comum)

PEÇA	POR QUE ESSA ESCOLHA
Node.js puro, sem framework	A máquina onde isso roda tem acesso restrito a instalar pacotes (política de TI corporativa). Node puro não depende de <code>npm install</code> nenhum.
PowerShell + Excel COM	Ler <code>.xlsx</code> sem biblioteca externa. O Excel já está instalado na máquina — COM (Component Object Model) é a forma nativa do Windows de controlar programas Office por script.
OpenXML puro para escrever o Excel de saída	Escrever no Excel via COM (<code>Range.Value2 = array</code>) trava com erro de memória em planilhas grandes. Gerar o <code>.xlsx</code> manipulando o XML compactado por dentro é mais chato de escrever, mas não trava.
Outlook COM para o <code>.oft</code>	É o único jeito de gerar um modelo de e-mail do Outlook clássico programaticamente no Windows.

ROTEIRO DAS FASES

1. Servidor Node mínimo
2. O config JSON — o cérebro do sistema
3. Ler os boletins .xlsx (PowerShell + Excel COM)
4. O motor de score
5. Escrever a planilha de saída (OpenXML puro)
6. A interface (cards, busca, filtros, dossiê para IA)
7. Gerar o modelo de e-mail .OFT (Outlook COM)
8. Empacotar, versionar e publicar no GitHub
9. Manter e calibrar depois de pronto

Fase 1 — Servidor Node mínimo

Objetivo: um servidor HTTP que sobe na porta 8790 e serve uma página HTML fixa. É a fundação sobre a qual tudo mais entra depois.

PROMPT PARA COLAR NA IA

Quero criar um app local em Windows chamado "NTSEC Radar". Comece pela base:

1. Crie a pasta do projeto com subpastas app/, config/, data/, scripts/.
2. Em app/server.js, escreva um servidor HTTP em Node.js PURO (módulo "http" nativo, sem Express nem nenhum pacote externo) que:
 - sobe na porta 8790
 - na rota GET / devolve um app/ui.html fixo (crie um ui.html simples de teste, so um "Ola Mundo", por enquanto)
 - imprime no console "Servidor rodando em http://localhost:8790"
3. Crie um arquivo Abrir-App.cmd na raiz que rode "node app\server.js" numa janela propria e depois abra o navegador em http://localhost:8790 automaticamente.

Depois de escrever, me diga exatamente que comando rodar para testar.

VERIFICAÇÃO

Rode o `Abrir-App.cmd`. Uma janela preta deve abrir e ficar parada — é o servidor rodando, não um travamento — e o navegador deve abrir sozinho mostrando a página de teste.

ATENÇÃO

Se a porta 8790 já estiver em uso (outro processo antigo), o Node erra com `EADDRINUSE`. Cole esse erro na IA — ela vai te ajudar a encontrar e fechar o processo antigo, ou trocar a porta.

Fase 2 — O config JSON (o cérebro do sistema)

Objetivo: o arquivo que descreve *o que a empresa vende*, para o motor de pontuação usar depois. Fazer essa fase antes do motor de pontuação é proposital: escrever o dado antes da lógica deixa claro o que a lógica precisa ler.

PROMPT PARA COLAR NA IA

Agora crie config/escopo-ntsec.json com esta estrutura (é o "cérebro" do sistema — a logica de pontuacao que vamos escrever depois so LE este arquivo, nunca tem numero fixo no código):

- "categorias": lista de objetos, cada um com "solucao" (nome da categoria, ex: "NGFW / Firewall / IPS"), "fabricantePadrao" (marca sugerida quando nao ha marca no texto) e "palavrasChave" (lista de termos em MINUSCULO E SEM ACENTO que, se aparecerem no texto do edital, indicam essa categoria).
- "fabricantes": lista de marcas que a empresa revende de verdade, cada uma com "nome", "solucao" (a categoria correspondente) e "aliases" (nomes alternativos/produtos da marca).
- "concorrentes": mesma ideia, mas marcas de concorrentes (uma citacao de concorrente e sinal de oportunidade de troca de fornecedor, nao de venda direta).
- "contextosForaEscopo": lista de termos que indicam OUTRA area de negocio (ex: mobiliario, obra civil, alimentacao) — servem para a pontuacao desconfiar quando aparecerem.
- "pesos": numeros de calibragem (quantos pontos cada sinal vale). NENHUM peso deve ficar fixo no codigo depois — tudo lido daqui.
- "faixas": as faixas de classificacao final (ex: "Prioridade Maxima" a partir de 75 pontos, "Alta" a partir de 55, e assim por diante), cada uma com "nome", "min" e "cor".

Popule com 3 ou 4 categorias de exemplo (ex.: Firewall, Backup, Rede) so para eu conseguir testar depois. Eu substituo pelo catalogo real da empresa mais tarde.

VERIFICAÇÃO

Abra o arquivo gerado num editor de texto e confira que é um JSON válido — sem vírgula sobrando no fim de lista, sem chave sem fechar. Se tiver dúvida, cole o conteúdo inteiro na IA e peça "confira se este JSON é válido".

Fase 3 — Ler os boletins .xlsx

Objetivo: um script PowerShell que abre os `.xlsx` baixados do Conclitação via Excel COM, identifica as colunas pelo **nome do cabeçalho** (não pela posição fixa — o boletim muda a ordem das colunas de tempos em tempos) e devolve uma lista de objetos com órgão, objeto, UF, data do pregão, valor estimado etc.

PROMPT PARA COLAR NA IA

Crie scripts/processar-boletins.ps1. Ele deve:

1. Ler todos os arquivos .xlsx da pasta data/boletins usando Excel COM (New-Object -ComObject Excel.Application), em modo invisível e só leitura.
2. Ler a LINHA 1 de cada planilha como cabeçalho, e montar um mapa "nome da coluna -> índice", procurando pelo NOME (com uma função FindCol que aceita um padrão de regex), não por número fixo de coluna - os boletins mudam a ordem das colunas ao longo do tempo.
3. Para cada linha de dado, montar um objeto com: numeroConclitacao, orgao, objeto, uf, numeroPregao, uasg, dataPregao, valorEstimado, situacao.
4. Manter só as linhas onde a Situacao seja "NOVA" (comparação sem diferenciar maiúsculas).
5. No fim, salvar tudo como JSON em data/dados-brutos.json.

Pontos IMPORTANTES de robustez, porque boletim real vem sujo:

- O arquivo .ps1 tem que ser salvo em ASCII PURO no código (comentários podem ter acento representado como [char]0x00E9 em vez de letra acentuada direta) - arquivo .ps1 com acento direto e sem BOM é lido errado pelo PowerShell 5.1 e corrompe silenciosamente.
- Ao converter texto de valor monetário para número, USE SEMPRE [double]::TryParse(\$texto, [Globalization.NumberStyles]::Float, [Globalization.CultureInfo]::InvariantCulture, [ref]\$var)
- nunca TryParse sem cultura explícita, porque em máquina com Windows configurado em português do Brasil o ponto decimal é interpretado como separador de milhar e o valor sai errado (silenciosamente, sem erro nenhum).
- Ao procurar a coluna de "Estado/UF" por regex de nome de cabeçalho, use um padrão que NAO bata também com "Valor Estimado" (que contém a palavra "Estimado", que começa com as mesmas letras "Est"). Prefira o padrão completo "estado" em vez de só "est".

Depois de escrever, rode o script contra um .xlsx de teste e me mostre o resultado.

VERIFICAÇÃO

Rode o script. Ele deve terminar sem erro e gerar `data\dados-brutos.json` com pelo menos uma licitação dentro, com todos os campos preenchidos corretamente — principalmente confira o **valor estimado** (compare com o que está escrito no boletim de verdade) e a **UF**.

ATENÇÃO

Estes dois bugs (cultura do TryParse e colisão de regex "Est") já aconteceram de verdade neste projeto: o valor estimado saía **10 vezes maior** que o real, e a UF saía em branco mesmo estando preenchida na planilha. Os dois só foram descobertos porque alguém comparou o número da tela com o número do boletim de verdade. Faça essa comparação manual pelo menos uma vez nesta fase.

Fase 4 — O motor de score

Objetivo: a função que lê o objeto de cada licitação e devolve uma nota de 0 a 100, com a explicação de cada ponto somado ou tirado. É a peça mais importante do sistema inteiro, e por isso vale a pena construir em subpassos, não de uma vez.

PROMPT PARA COLAR NA IA

Agora, no mesmo scripts/processar-boletins.ps1, adicione uma função AvaliarItem(\$item) que calcula um score de 0 a 100. Construa em camadas, cada uma somando ou subtraindo do total, e GUARDANDO em uma lista "porques" o nome do sinal e quantos pontos ele deu (isso é essencial: o usuário final precisa ver o PORQUE da nota, não só o número):

CAMADA 1 - Casamento de categoria: normalize o texto do objeto (minúsculo, sem acento, pontuação vira espaço) e procure cada palavraChave do config/escopo-ntsec.json como PALAVRA INTEIRA (com espaço antes e depois, não substring solta - "pam" não pode casar dentro de "equipamento").

Cada categoria encontrada soma pontos (config "pesos.categoriaPrimeira" para a primeira, menos para as seguintes).

CAMADA 2 - Marca parceira ou concorrente: se o nome ou alias de uma marca do config aparecer no texto, soma pontos (marca parceira vale mais que concorrente).

CAMADA 3 - Termo de outra area: se 2 ou mais termos de "contextosForaEscopo" aparecerem E formarem a MAIORIA dos sinais encontrados (nao so uma contagem fixa), SUBTRAI pontos - a ideia e reconhecer quando o edital e de outro ramo e so tem 1-2 palavras nossas por acaso.

CAMADA 4 - Prazo: calcule quantos dias faltam para a data do pregao. Prazo confortavel soma pontos pequenos, prazo vencido subtrai bastante - MAS SO quando ja existe alguma aderencia tecnica (camada 1 encontrou algo). Nao faz sentido premiar prazo de um edital que nao e nosso.

No final, classifique em uma FAIXA (config "faixas") de acordo com o total.

Depois de escrever, rode contra os dados de teste da Fase 3 e me mostre o score e o "porques" de cada licitacao.

VERIFICAÇÃO

Pegue uma licitação de teste cujo texto você já leu com seus próprios olhos. O score bate com o que você julgaria manualmente? A lista de "porquês" faz sentido, e cada item ali realmente aparece no texto do objeto?

ATENÇÃO

Uma palavra genérica (como "resposta", "auditoria" ou siglas de dois-três caracteres) pode achar "sinal" em texto que não tem nada a ver com o que a empresa vende. Neste projeto, cerca de 30% dos falsos positivos vieram de palavra solta demais no dicionário. Sempre que adicionar uma palavra nova, teste-a contra vários exemplos de texto antes de confiar nela.

Fase 4b — As camadas de precisão (construir só depois que a Fase 4 estiver funcionando)

As quatro camadas acima classificam razoavelmente bem, mas **o motor real deste projeto tem 13 sinais distintos**, não 4. Os outros 9 só nasceram porque casos reais, com dados reais, expuseram erros que a versão simples não via. Este manual não finge que dá para adivinhar esses refinamentos de antemão: eles vieram de meses de calibragem contra editais de verdade. O que dá para ensinar é o **mecanismo** de cada um — os números exatos de calibragem (quantos pontos, qual limiar) são específicos dos dados desta empresa e devem ser recalibrados com os seus próprios exemplos, nunca copiados como se fossem universais.

PROMPT PARA COLAR NA IA

Adicione a AvaliarItem() estas camadas extras, cada uma como um refinamento separado e testavel:

5. TERMOS PROIBIDOS x CONTEXTO DE OUTRA AREA - sao dois mecanismos diferentes, nao um so:
 - "termosExcluidos" no config: penalidade FIXA e forte quando um termo aparece (ex.: um termo que so existe em outro ramo de negocio completamente, tipo "merenda escolar").
 - "contextosForaEscopo": penalidade PROPORCIONAL (a da Camada 3) - dispara por RAZAO entre sinais de fora e sinais de dentro, nao por contagem fixa.
 Motivo de separar: alguns termos sao prova definitiva sozinhos; outros so viram prova quando dominam o texto.
6. PORTE DO CONTRATO: se o boletim trouxer um valor estimado, some pontos por faixa de valor (CONTRATO grande desempata dentro da mesma faixa tecnica) - MAS SO quando ja ha aderencia tecnica minima. Sem essa trava, um contrato gigante de outra area (ex: obra civil que so cita "datacenter" de passagem) ganha pontos por tamanho sem ser nosso.
7. PROJETO INTEGRADO: bonus extra quando 2+ categorias do portfolio aparecem no MESMO edital (e o sinal for denso, nao disperso num texto gigante) - indica solucao integrada, nao coincidencia. NAO aplique esse bonus quando o edital for Registro de Precos (ver item 9) - la, varias categorias so significam "cesta variada de produtos", nao projeto integrado.
8. ESPECIFICIDADE DA PALAVRA: no config, separe as palavras-chave de cada categoria em duas listas extras: "nucleo" (termo que so existe no seu ramo, sem ambiguidade - ex: um nome de

categoria de produto de segurança bem específico) e "ambíguas" (palavra que também aparece em textos de outra coisa - ex: uma palavra genérica de TI que serve pra várias áreas). Palavra de núcleo multiplica os pontos da categoria para cima; palavra so-ambígua sozinha multiplica para baixo (ela sozinha não prova nada, mas conta se vier ACOMPANHADA de outra palavra da mesma categoria - duas ambíguas juntas da mesma área formam uma expressão, como "backup" + "armazenamento" apontando pra a mesma solução).

9. A ABERTURA DO OBJETO DECLARA O QUE SE COMPRA: o edital sempre diz o que está comprando nos primeiros caracteres do objeto, e detalha depois. Extraia só a ABERTURA (por exemplo os primeiros 150-200 caracteres, sem contar prefixos padrão tipo "* Licitação Eletrônica *") e verifique se ela tem alguma palavra ESPECÍFICA nossa (não ambígua) ou marca parceira. Se a abertura só tem termo de OUTRA área e nada nosso, os pontos de categoria ganhos por palavra ambígua no resto do texto devem ser CANCELADOS (não só multados com penalidade fixa) - senão um contrato de outra área com vocabulário ambíguo espalhado ainda passa.

10. REGISTRO DE PREÇOS (RP): quando o boletim marcar o edital como Registro de Preços e ele vier dividido em itens/lotos, a proporção de itens que são nossos NÃO deve reduzir a aderência técnica do mesmo jeito que reduziria numa contratação de lote único - em RP o comprador pode fechar só nos itens que quiser. Em vez disso, aplique um DESCONTO GRADUADO proporcional ao tamanho da fatia (fatia pequena vale menos esforço de disputa, mas não e fora de escopo).

Depois de cada uma, rode contra os MESMOS exemplos de teste e confirme que nada que estava certo ficou errado - anote a nota de cada exemplo antes e depois de cada camada nova.

VERIFICAÇÃO

Depois de adicionar as camadas 5 a 10, rode a lista completa de exemplos de teste de novo e compare com a rodada anterior. Alguma nota que estava certa mudou para pior? Se sim, a camada nova está grande demais para o caso — ela deveria só ter mexido nos casos que a motivaram.

ATENÇÃO

Cada camada nova deve ser motivada por um **caso real classificado errado**, nunca adicionada "por precaução". Uma regra nova generalizada de um único exemplo é frágil: neste projeto, uma regra criada a partir de um caso só teve que ser revertida na rodada seguinte porque o próprio caso que a motivou acabou sendo, na verdade, uma exceção.

Fase 5 — Escrever a planilha de saída

Objetivo: gerar um `.xlsx` com o resultado sem usar Excel COM para escrever, porque esse caminho trava com erro de memória em planilhas grandes. A saída é manipular o XML compactado diretamente.

PROMPT PARA COLAR NA IA

Adicione a `scripts/processar-boletins.ps1` uma função que grava o resultado como `.xlsx` usando OpenXML puro (`System.IO.Packaging`), NÃO Excel COM (`Range.Value2 = array` trava com erro de memória em planilhas com muitas linhas). A planilha deve ter:

- Aba "Ranking": órgão, objeto (resumido), score, faixa, link - ordenada do maior score para o menor.
- Cabeçalho congelado (freeze panes) na primeira linha.
- A célula de score colorida de acordo com a cor da faixa (config `"faixas[.].cor"`).
- Uma coluna de link como hyperlink de verdade clicável (célula de fórmula `HYPERLINK` precisa do atributo `t="str"` no XML, senão o Excel recusa abrir o arquivo - e o erro só aparece ao ABRIR o arquivo depois, o XML fica bem formado e não acusa erro na hora de gravar).

Rode e abra o `.xlsx` gerado no Excel de verdade para confirmar que abre sem erro de reparo.

VERIFICAÇÃO

Abra o `.xlsx` gerado no Excel de verdade (não só confie que "não deu erro no script"). Se o Excel disser algo como "encontramos um problema com algum conteúdo" e oferecer para reparar, a fase não está pronta — cole essa mensagem do Excel na IA.

Fase 6 — A interface

Objetivo: substituir a página de teste da Fase 1 pela interface real: lista de cartões, busca, filtros e o botão de montar o dossiê para colar numa IA.

PROMPT PARA COLAR NA IA

Substitua app/ui.html por uma pagina real de uma unica tela (HTML + CSS + JavaScript, sem framework, tudo num arquivo so). Ela deve:

1. Ao carregar, buscar os dados processados (crie uma rota GET /api/dados no server.js que devolve o conteudo de data/dados.json).
2. Mostrar cada licitacao como um "cartao": orgao, score grande, faixa colorida, objeto (truncado com botao "ver mais"), lista de "porques" com os pontos de cada sinal.
3. Uma barra de busca no topo que filtra por orgao, objeto, fabricante OU numero do pregao/Conlicitacao - aceitando o numero com ou sem pontuacao (19280729 e 19.280.729 tem que achar a mesma licitacao).
4. Filtros por faixa (botoes: Todas / Parceiros / Concorrentes / Escopo Geral / Baixa).
5. Um botao "Processar boletins" que chama uma rota POST /api/processar no server.js, que roda o processar-boletins.ps1 via child_process e recarrega a lista quando terminar.
6. Um botao "Copiar dossie para IA" que monta um texto grande juntando: uma instrucao fixa explicando o que a IA deve avaliar, o config/escopo-ntsec.json inteiro (para a IA saber o que a empresa vende) e os dados de cada licitacao SELECIONADA (com checkbox em cada cartao) - e copia esse texto para a area de transferencia.

Use uma paleta de cores sobria (tons terrosos, sem neon) definida em variaveis CSS no :root, e deixe pronto para um modo escuro por cima das mesmas variaveis.

VERIFICAÇÃO

Clique em cada botão da interface pelo menos uma vez. Busque por um número de pregão com e sem pontuação e confirme que os dois formatos acham a mesma licitação. Copie o dossiê e cole num editor de texto qualquer para conferir que veio completo.

ATENÇÃO

Um único caractere de escape errado dentro de um template literal do JavaScript (por exemplo um apóstrofo mal escapado dentro de uma string entre crases) derruba o `<script>` inteiro. A página carrega, mas **nenhum** botão funciona, e não aparece erro nenhum visível na tela. Se isso acontecer, peça à IA para "verificar a sintaxe do JavaScript dentro do ui.html" — ela consegue rodar um verificador de sintaxe para achar a linha exata.

Fase 7 — Gerar o modelo de e-mail .OFT

Objetivo: um botão que baixa um arquivo `.oft` pronto para abrir no Outlook, com o conteúdo já preenchido, sem enviar nada e sem abrir o Outlook na tela da pessoa.

PROMPT PARA COLAR NA IA

Crie scripts/gerar-oft.ps1: recebe uma lista de licitacoes (via um arquivo JSON temporario) e monta o corpo de um e-mail em HTML com os dados de cada uma. Use Outlook COM (New-Object -ComObject Outlook.Application) para criar um item de e-mail novo (nao enviado, nao aberto na tela) e salve-o com metodo SaveAs(caminho, 2) - o "2" e o codigo que salva como modelo .oft (OlSaveAsType.olTemplate), diferente de salvar como .msg.

No server.js, crie uma rota POST /api/gerar-oft que recebe as licitacoes selecionadas, roda este script e devolve o arquivo .oft para download.

IMPORTANTE: nao use CreateItemFromTemplate nem qualquer metodo que ABRA a janela do Outlook ou do e-mail na tela do usuario - o objetivo e so gerar o arquivo, silenciosamente, sem interromper quem esta usando o computador.

VERIFICAÇÃO

Baixe o `.oft` gerado e dê duplo clique nele. Deve abrir uma janela de e-mail **nova** no Outlook, com o conteúdo já preenchido, pronta para editar o destinatário e enviar manualmente. O sistema nunca deve enviar sozinho.

Fase 8 — Empacotar, versionar e publicar

PROMPT PARA COLAR NA IA

Crie:

1. Um `.gitignore` que exclua dados reais (`data/boletins/*.xlsx`, `data/dados.json`, `config/contatos.json`) mas mantenha o código e a estrutura de pastas (usando um arquivo `.gitkeep` dentro de pastas que precisam existir vazias, já que o git não versiona pasta vazia).
2. Um `scripts/publicar-versao.ps1` que: confere que nada da lista proibida está na fila do git, faz commit, cria uma tag de versão (`v1.0.X`, incrementando sozinho), empurra para o GitHub e cria uma Release (página de download) via API do GitHub.
3. Um `INSTALACAO.md` explicando para quem nunca usou git como baixar e rodar pela primeira vez.

No `publicar-versao.ps1`, NÃO use `"git push --tags"` (empurra todas as tags de uma vez e uma tag antiga com conflito reprova o push inteiro) - empurre só a tag da vez. E não use `$ErrorActionPreference = 'Stop'` junto de comando git (o git escreve aviso normal no `stderr` e isso vira erro fatal sem motivo).

VERIFICAÇÃO

Rode o script de publicação uma vez. Confirme no site do GitHub que apareceu a nova versão, e baixe o `.zip` da Release para confirmar que os arquivos certos vieram (e nenhum dado sensível).

Fase 9 — Depois de pronto: manter e calibrar

O sistema não termina na Fase 8. O dicionário de palavras-chave precisa de ajuste contínuo conforme editais reais aparecem. A forma que funcionou aqui: sempre que uma licitação estiver classificada errado, pergunte à IA qual **propriedade geral** do texto separa esse caso dos que estão certos, em vez de corrigir só aquele caso. E sempre meça o efeito da mudança contra uma lista grande de licitações antes de publicar, não só contra o caso que motivou a mudança.

NTSEC Radar — Manual de Construção, Radar de Boletins. Este roteiro reflete a ordem real em que o sistema foi construído e as armadilhas que realmente aconteceram durante a construção. Não é um roteiro teórico.